

“Hospital Emergency Patient Monitoring System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

**SAI ANANYA ROY
D SHREENITHA**

**2520030471
2520030504**

Under the Esteemed Guidance of

Dr. K. Anusha

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

**K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled “Hospital Emergency Patient Monitoring System” is a record of Bonafide work of SAI ANANYA ROY - 2520030471, D SHREENITHA - 2520030504 submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled“Hospital Emergency Patient Monitoring System” is a bonafide work done and submitted by SAI ANANYA ROY – 2520030471 and D SHREENITHA – 2520030504 ,

in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2024-2025**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr. K. Anusha**, and my guide **Dr. K. Anusha**, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)**, Head of the *Department for Computer Science Engineering* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Abstract

The Hospital Emergency Patient Monitoring System is a Linux-based academic project that demonstrates Operating System concepts through a practical hospital emergency scenario. The system collects basic patient vital information such as Patient ID, Heart Rate, Blood Pressure, Temperature, and Oxygen Level. The collected data is transferred between independent processes using Linux Inter-Process Communication (IPC) through named pipes (FIFOs).

The Patient Monitoring Process accepts the patient information and sends it through the `patient_pipe` FIFO to the Emergency Processor. The Emergency Processor reads the received `PatientData`, checks the vital signs using predefined project thresholds, and classifies the patient status as `NORMAL`, `WARNING`, or `EMERGENCY`. The system also records the patient information, status, condition, and recommended action in `alerts.log`.

A simple web frontend is also provided with four pages: Login, Patient Details, Patient Result, and Doctor Alert. The interface allows the user to enter patient data and view the monitoring result and doctor alert. The project demonstrates process communication, named pipes, Linux system calls, file handling, process coordination, and basic web-interface integration. The thresholds used in this project are simplified academic rules and are not medical guidelines.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		1–5
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		6–12
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		13–17
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		18–22
		4.1 C Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 Named Pipes / FIFO	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	
5	System Architecture and Design		23–29
		5.1 Overall System Architecture	
		5.2 System Workflow	
		5.3 Flowchart / Data Flow	
		5.4 Module Description	

		5.5 Input and Output Design	
6	Implementation		30– 45
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C / C++ Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 File and Resource Management	
		6.8 Optional User Interface Implementation	
7	Execution and Demonstration		62– 67
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	Conclusion		79– 83
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	References		84– 86
		9.1 Books	
		9.2 Research Papers	
		9.3 Linux / GNU Documentation	
		9.4 Online Resources	

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Hospitals need quick access to patient vital information, especially when a patient may require immediate attention. This project models a simple hospital monitoring workflow using Linux processes and Inter-Process Communication. The system collects Patient ID, Heart Rate, Blood Pressure, Temperature, and Oxygen Level and transfers the information from one process to another.

1.2 Problem Statement

The main problem addressed is how patient information can be transferred between independent Linux processes and analyzed automatically to generate a clear patient status and alert. The project demonstrates this communication using named pipes and a rule-based health classification.

1.3 Scope of the Project

The project covers patient data collection, IPC using named pipes, basic vital-sign analysis, alert logging, and a simple web frontend. It is designed as an academic Operating Systems demonstration and is not intended for real medical diagnosis.

1.4 Objectives of the Project

- To demonstrate Linux Inter-Process Communication using named pipes.
- To transfer patient vital data from the monitoring process to the Emergency Processor.
- To classify patient data as NORMAL, WARNING, or EMERGENCY using predefined project rules.
- To maintain an alert log containing patient and monitoring information.
- To provide a simple frontend for patient entry, result display, and doctor alert display.

1.5 Importance of the Application

The project connects Operating System concepts with a practical hospital scenario. It shows how separate processes can exchange structured data, process the received information, and prepare an alert while maintaining a record of the result.

1.6 Target Users / Audience

The main audience is students and faculty evaluating Operating Systems and Linux IPC concepts. The frontend also represents how a simple hospital monitoring interface could be presented to a user.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Operating System Concepts Related to the Project

The project demonstrates processes, Inter-Process Communication, file descriptors, named pipes (FIFOs), file handling, and Linux system calls. The Patient Monitoring Process and Emergency Processor are separate programs that communicate through `patient_pipe`.

2.2 Existing Methods / Algorithms

A manual approach would require a person to enter patient information and communicate the result manually. In this project, the communication is automated between processes. A simple rule-based threshold algorithm is used to classify the patient as NORMAL, WARNING, or EMERGENCY.

2.3 Proposed Approach

The proposed system uses the Patient Monitoring Process to collect patient information and send a PatientData structure through `patient_pipe`. The Emergency Processor reads the structure, checks the vital signs, displays the result, writes the information to `alerts.log`, and prepares an

alert message for hospital_fifo.

2.4 Advantages of the Proposed Approach

The approach is simple, easy to demonstrate, and clearly shows Linux IPC. It separates data collection from data analysis, keeps an alert record, and can be extended later with additional processes, remote monitoring, or a database.

3.1 Hardware Requirements

A computer or laptop with sufficient RAM and storage is required. A keyboard and display are used for entering and viewing patient information.

3.2 Software Requirements

Ubuntu Linux or Ubuntu through WSL, GCC compiler, VS Code, a web browser, and a Linux/POSIX C environment are required.

3.3 Operating System Environment – Ubuntu / WSL

The C programs use Linux/POSIX headers and FIFO operations such as mkfifo(), open(), read(), write(), and close(). Therefore, Ubuntu or Ubuntu through WSL is used to compile and execute the C programs.

3.4 Development Tools and Compilers

VS Code is used for source-code editing and frontend development. GCC is used to compile the C programs. Live Server can be used to test the HTML/CSS/JavaScript frontend.

CHAPTER -3 TECHNOLOGY AND TOOLS USED

4.1 C Programming

C is used to implement the Patient Monitoring Process and Emergency Processor.

4.2 GCC Compiler

GCC compiles main.c and emergency.c into executable programs in the Linux/WSL environment.

4.3 Linux System Calls

Important Linux/POSIX functions used include mkfifo(), open(), read(), write(), close(), unlink(), fopen(), fprintf(), and fclose().

4.4 Named Pipes / FIFO

patient_pipe transfers PatientData from the Patient Monitoring Process to the Emergency Processor. hospital_fifo is prepared as the communication path for the processed doctor alert.

4.5 Linux Commands and Utilities

Commands such as gcc, ls, cd, rm, and program execution commands are used during compilation, file management, and testing.

4.6 Other Tools / Libraries Used

The frontend uses HTML, CSS, and JavaScript. Standard C libraries and Linux/POSIX headers are used in the C programs.

CHAPTER -4 SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

User / Frontend → Patient Monitoring Process → patient_pipe → Emergency Processor → Health Analysis → alerts.log and hospital_fifo → Doctor Alert.

5.2 System Workflow

The user logs in, enters Patient ID and vital signs, and starts monitoring. The patient data is sent through patient_pipe. The Emergency Processor reads and analyzes the data, classifies the

patient, logs the result, and prepares the doctor alert.

5.3 Flowchart / Data Flow

Login → Patient Details → Patient Monitoring → patient_pipe → Emergency Processor → NORMAL/WARNING/EMERGENCY → Alert Log → Doctor Alert.

5.4 Module Description

Patient Monitoring Module: collects patient data. Emergency Processor: analyzes the data.

Logging Module: stores the result in alerts.log. Doctor Alert Module: represents the alert communication stage. Frontend Module: provides Login, Patient Details, Result, and Doctor Alert pages.

5.5 Input and Output Design

Inputs are Patient ID, Heart Rate, Blood Pressure, Temperature, and Oxygen Level. Outputs include patient status, possible condition, recommended action, monitoring time, and doctor alert information.

CHAPTER -5 IMPLEMENTATION

6.1 Module-wise Implementation

main.c implements the Patient Monitoring Process. emergency.c implements the Emergency Processor, health classification, logging, and hospital FIFO alert preparation. The frontend files provide the user interface.

6.2 Algorithm / Pseudocode

BEGIN → Create/open patient_pipe → Read patient details → Write PatientData → Emergency Processor reads data → Check emergency rules → Check warning rules → Otherwise NORMAL → Display and log result → Prepare doctor alert → Repeat/END.

6.3 Core C Implementation

Both C programs use the same PatientData structure containing Patient ID, Heart Rate, systolic BP, diastolic BP, Temperature, and Oxygen Level.

6.4 Linux System Calls / APIs Used

mkfifo() creates named pipes. open() opens the FIFO. read() receives data. write() sends data. close() releases file descriptors. unlink() can remove FIFO files. fopen(), fprintf(), and fclose() are used for alerts.log.

6.5 Process Implementation

The project uses two main C processes: Patient Monitoring Process and Emergency Processor. They perform separate tasks and communicate through a FIFO.

6.6 IPC / Synchronization Implementation

patient_pipe is the main IPC channel between the two C programs. The Emergency Processor waits for patient data, while the Patient Monitoring Process writes the PatientData structure. hospital_fifo is used for the next alert communication stage.

6.7 File and Resource Management

The programs check errors while creating and opening FIFOs, close file descriptors after communication, and append monitoring results to alerts.log.

6.8 Optional User Interface Implementation

The frontend contains index.html for login, patient.html for patient details, result.html for the monitoring result, doctor.html for the doctor alert, style.css for design, and script.js for page navigation and frontend logic.

CHAPTER -6 EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

```
gcc main.c -o main
gcc emergency.c -o emergency
```

7.2 Execution Commands

Run the Emergency Processor first using ./emergency. In another terminal, run the Patient Monitoring Process using ./main.

7.3 Sample Inputs and Outputs

NORMAL: ID 2, Heart Rate 75 BPM, BP 120/80 mmHg, Temperature 36 C, Oxygen 98% →
NORMAL.

WARNING: ID 3, Heart Rate 105 BPM, BP 125/80 mmHg, Temperature 37 C, Oxygen 94% →
WARNING.

EMERGENCY: ID 4, Heart Rate 130 BPM, BP 150/95 mmHg, Temperature 40 C, Oxygen 88%
→ EMERGENCY.

7.4 Screenshots of Execution

Screenshots of the C program output, alerts.log, and the four frontend pages can be inserted in this section.

7.5 Demonstration of OS Concepts

The demonstration shows named-pipe creation, inter-process data transfer, file descriptors, read/write operations, process separation, and alert logging.

CHAPTER -7 CONCLUSION

8.1 Summary of the Project

The Hospital Emergency Patient Monitoring System demonstrates Linux process communication using named pipes in a practical hospital scenario. Patient data is collected, transferred, analyzed, logged, and prepared for alert communication.

8.2 Objectives Achieved

Patient data collection, named-pipe IPC, rule-based status classification, alert logging, and a four-page frontend were implemented.

8.3 Results Achieved

The system produces NORMAL, WARNING, and EMERGENCY results for the selected sample inputs and demonstrates communication between the monitoring and emergency processes.

8.4 Skills and Learning Outcomes

The project provided practical learning in C programming, Linux system calls, FIFO-based IPC, process communication, GCC compilation, debugging, and basic HTML/CSS/JavaScript frontend development.

CHAPTER -8 REFERENCES

9.1 Books

Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts.

9.2 Research Papers

Academic references related to Operating Systems and Inter-Process Communication may be consulted for further study.

9.3 Linux / GNU Documentation

Linux manual pages for mkfifo, open, read, write, close, unlink, and related POSIX APIs.

9.4 Online Resources

Ubuntu, GNU GCC, and Visual Studio Code documentation.